One Demonstration Is Enough for Real-World Robotic Reinforcement Learning

Anonymous Authors

Anonymous Institution

Abstract. Learning effective robot control policies on physical hardware is challenging due to costly data collection and the difficulty of reward specification. Prior work has incorporated demonstrations into reinforcement learning (RL), yet existing approaches either require large numbers of demonstrations or depend on continuous human supervision during training. We present *AutoSERL*, a framework that leverages a single human demonstration to fully automate the human intervention process in real-world robot RL. Our key insight is that one demonstration trajectory encodes sufficient structural information to define three complementary mechanisms: a *sliding window intervention* that continuously guides exploration to prevent local optima and unsafe deviations, a *safety recovery mechanism* that detects and corrects failure states via predefined trajectory recovery points, and an *intervention termination* criterion that automatically disables guidance once the policy can independently complete the task, preserving its exploration advantage. We evaluate AutoSERL on six contact-intensive manipulation tasks across two robot platforms, spanning insertion, hanging, and hinge-based tasks. AutoSERL consistently outperforms SERL initialized with 20 demonstrations, behavior cloning, and MILES — a dedicated one-shot imitation learning baseline — across all tasks, achieves 100% success rate on insertion tasks, and demonstrates improved robustness to positional variations, all from a single demonstration.

Keywords: Robotic Reinforcement Learning · One-shot Learning · Sample Efficiency

1 Introduction

Reinforcement learning (RL) has demonstrated remarkable success in simulated environments [11,16], yet its deployment on physical robotic systems remains fundamentally constrained by two persistent challenges. First, real-world exploration carries inherent safety risks: unlike simulation where failures are costless, uncontrolled robot motions can cause hardware damage or environmental collisions [2]. Second, the sample inefficiency of model-free RL means that agents often fail to encounter the sparse reward signals necessary for policy convergence, particularly in contact-intensive tasks where successful states occupy only a small fraction of the state space.

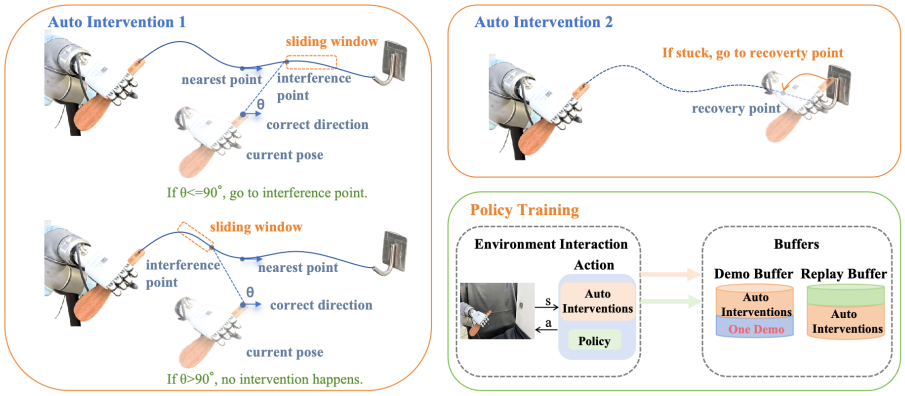

Fig. 1. Overview of AutoSERL. **Auto Intervention 1** (Sliding Window Intervention): the robot is guided to the nearest point within the sliding window only when the angle θ between the trajectory's forward direction and the vector to that point satisfies $\theta \leq 90°$, preventing the robot from being pulled back to already-visited positions. **Auto Intervention 2** (Safety Recovery Mechanism): when the robot is stuck, it is guided to the recovery point and the demonstration segment is replayed to restore progress. **Policy Training**: intervention-guided transitions and the single human demonstration are stored in the Demo Buffer and Replay Buffer for policy training.

To address these challenges, recent work has proposed combining demonstrations with RL to accelerate real-world learning [17,14,12]. [8] introduces a sample-efficient framework that mixes demonstration data with online experience, significantly improving training stability on physical hardware. Building upon this, [9] incorporates human-in-the-loop intervention during training, where a human operator monitors the robot in real time and teleoperates it out of deadlocks or unsafe states. The corrected trajectories are added to the replay buffer, providing high-quality guidance that helps overcome sparse rewards and prevents unsafe exploration. This paradigm has proven effective for learning precise manipulation policies directly on real robots.

However reliance on continuous human supervision introduces a critical scalability bottleneck. Human operators are subject to cognitive fatigue, inconsistent response times, and high labor costs per training hour. As tasks grow more complex and training durations extend, the requirement for constant human presence becomes logistically impractical. This raises a fundamental question: *can the benefits of human-in-the-loop training be preserved while eliminating the need for continuous human involvement?*

In this paper, we present *AutoSERL*, a framework that replaces continuous human supervision with automated intervention mechanisms derived from a **single human demonstration**. Our key insight is that one demonstration trajectory encodes sufficient structural information about task geometry and contact sequences to serve as a complete guidance signal throughout training.

Concretely, AutoSERL consists of three complementary components. A *sliding window intervention* mechanism continuously monitors the agent's end-effector pose relative to a window sliding along the demonstration trajectory, detecting deviations caused by local optima, Q-value overestimation, or environmental obstacles, and redirecting the robot accordingly. A *safety recovery mechanism* detects stagnation near interaction objects and triggers a replay-based recovery by re-executing a predefined segment of the demonstration trajectory, restoring the robot to a productive state without human involvement. An *intervention termination* criterion monitors intervention frequency during each episode and automatically disables all guidance once the policy demonstrates sufficient autonomy, allowing the agent to fully leverage RL's exploratory advantage without unnecessary constraint.

We evaluate AutoSERL on six contact-intensive manipulation tasks spanning insertion, hanging, and hinge-based categories across two robot platforms. AutoSERL consistently outperforms [8] initialized with 20 demonstrations, behavior cloning (BC), and MILES [13] — a dedicated one-shot imitation learning method — across all tasks, achieves 100% success rate on insertion tasks, and maintains strong performance under positional variations, all from a single demonstration. Our results demonstrate that one demonstration, when carefully structured into an automated intervention system, is sufficient to match and exceed the effectiveness of both multi-demonstration RL and human-supervised training.

The main contributions of this paper are as follows. 1) we propose AutoSERL, a framework that automates the human intervention process in real-world robot RL using only a single demonstration, eliminating the need for continuous human supervision; 2) we design three complementary mechanisms — sliding window intervention, safety recovery, and intervention termination — that together form a closed-loop automated guidance system derived entirely from one demonstration trajectory; 3) we conduct extensive real-world experiments on six contact-intensive manipulation tasks across two robot platforms, demonstrating that AutoSERL consistently outperforms multi-demonstration RL, behavior cloning, and one-shot imitation learning baselines.

2 Related Works

2.1 Human-in-the-Loop RL

A prominent strategy to address the challenges of real-world exploration and sparse reward signals is to integrate human expertise directly into the training loop through real-time interventions. Paradigms such as Interactive Learning[1,7,10,18] and DAgger-style imitation[19,3,6,15] allow human supervisors to provide corrective feedback or take over control when the agent enters a hazardous or non-productive state. By injecting this expert-induced bias, these methods transform random exploration into a structured data collection process, significantly accelerating policy convergence on physical hardware. However, these methods suffer from a "human bottleneck." The need for constant vigilance creates a scalability

paradox: while humans ensure safety, their presence restricts training duration and scale due to cognitive fatigue, latency, and high costs. Instead, our Auto-Intervention framework achieves the benefits of corrective supervision without the need for a human in the loop.

2.2 Safety and Exploration in Real-World RL

An established approach to ensuring safety during real-world exploration is to employ Constrained Markov Decision Processes (CMDPs) or protective safety shields that filter out hazardous actions based on predefined constraints[4,**?**,**?**,**?**,**?**,**?**]. By enforcing strict operational boundaries, these methods effectively safeguard hardware against catastrophic collisions during the learning process. However, such systems often require complex multi-robot coordination and specific hardware setups. Instead, our method achieves similar levels of autonomy and safety through purely rule-based geometric heuristics, requiring no additional robotic "teacher".

2.3 Learning from Demonstration Replay

A highly efficient approach to accelerating policy acquisition is to utilize replay-based imitation learning, which estimates the robot's pose relative to objects of interest and re-executes demonstrated action sequences[5,**?**,**?**,**?**]. By leveraging existing expert trajectories, these methods provide a strong initial bias that bypasses the need for exhaustive random exploration in high-dimensional state spaces. Instead, our approach utilizes a replay-based recovery protocol that leverages a sliding window of the agent's own successful historical trajectories. By re-executing these validated segments upon detecting stagnation or interaction deadlocks, our framework provides a computationally efficient and robust guidance mechanism.

3 Method

3.1 Preliminaries

Reinforcement learning (RL) tasks can be modeled as a Markov Decision Process $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \rho, P, r, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\rho(s_0)$ is the initial state distribution, P is the state transition probability, r is the reward function, and γ is the discount factor. The learning objective is to maximize the expected cumulative reward. Our method is built upon SERL [8].

3.2 Automated Intervention Design

While SERL significantly improves sample efficiency, training can still fail on tasks with high difficulty and sparse rewards. To address this, [9] proposed HIL-SERL, which extends SERL by introducing human interventions during training

to correct the robot's actions, thereby alleviating the negative impact of sparse rewards and enabling faster and more stable learning. However, HIL-SERL requires continuous human involvement throughout training, resulting in high labor costs. To reduce this burden, we analyze the situations in which human intervention is required in HIL-SERL, and use these observations as the basis for designing AutoSERL.

Taking tasks that involve interaction between a hand-held object and a single target object as an example, we identify three principal situations in which intervention is needed. The first situation occurs when the training process falls into a local optimum, where the magnitude of the output actions gradually decreases and the robot arm keeps oscillating around a certain position. The second situation occurs when the Q-value is incorrectly estimated during training, causing the policy to continuously output actions that move the robot away from the target object, which significantly reduces learning efficiency. The third situation occurs when reinforcement learning is exploring normally, but obstacles exist in the real-world environment, potentially leading to collisions with surrounding obstacles or causing the robot to get stuck. Beyond these three situations, a residual risk exists: even when none of the above conditions are met, the robot may still become physically stuck near an obstacle and fail to make further progress. These four risks collectively define the design requirements for AutoSERL.

3.3 AutoSERL

AutoSERL uses a single human demonstration trajectory as guidance and addresses the identified risks through three complementary mechanisms, as illustrated in Fig. 1. Before training begins, a demonstration trajectory is manually collected, and two recovery points are defined on it: *recover point*$_0$, a safe recovery target to which the robot can be guided when a safety risk occurs, and *recover point*$_1$, a trajectory point at which the robot is in stable contact with the interaction object. These two points can be identified by replaying the demonstration trajectory on the robot and serve as shared references across all three mechanisms.

The first and second situations—local optimum convergence and erroneous Q-value estimation—along with the third situation involving environmental obstacles, are all handled by the *Sliding Window Intervention* mechanism, which actively guides the robot along the demonstration trajectory at every time step; since the trajectory is collected while maintaining a safe distance from obstacles, it naturally encodes obstacle avoidance. The residual stagnation risk is addressed by the *Safety Recovery Mechanism*, a passive fallback that detects when the robot is stuck and restores it to a safe state. Finally, the *Intervention Termination* mechanism disables all interventions once the policy becomes capable of completing the task independently, preventing continuous intervention from suppressing exploration.

All tasks considered in this paper involve interaction between a hand-held object and a single target object, and the entire method is built upon and extended from this demonstration trajectory.

Sliding Window Intervention To leverage the demonstration trajectory for guiding the training process, we further define a sliding window. The length of the sliding window is equal to the index difference between *recover point*$_0$ and *recover point*$_1$ on the demonstration trajectory. The window contains a continuous segment of trajectory indices. At each time step, the system computes the distance between the current end-effector pose and all end-effector poses on the demonstration trajectory within the sliding window. The trajectory point with the minimum distance is selected and referred to as a potential intervention point *pipoint*. When the minimum distance exceeds the intervention threshold th_2, further judgment is required to determine whether intervention should be performed.

To avoid pulling the robot back to past trajectory positions, a directional check is conducted on the *pipoint*. Let v_1 denote the tangent vector of the trajectory at the current *cpoint*, and let v_2 denote the vector from the current end-effector pose to the *pipoint*. The angle between these two vectors is computed. If the angle is greater than 90°, the *pipoint* is considered to lie on a past segment of the trajectory, and the intervention is discarded. Otherwise, motion planning is used to move the robot to the *pipoint*.

At initialization, the end point of the sliding window is located at the first point of the demonstration trajectory. When no intervention occurs, the sliding window moves forward along the demo trajectory by one step at each time step. If the starting point of the sliding window reaches the end of the trajectory, the portion of the window that has reached the trajectory end will remain fixed, while the remaining portion that has not yet reached the end will continue to move forward. Therefore, using the sliding window for intervention can effectively prevent two situations: (1) the robot oscillating around a fixed position due to convergence to a local optimum; and (2) the policy continuously outputting actions that move the robot away from the interaction object due to inaccurate Q-value estimation. Moreover, since the demonstration trajectory is collected while maintaining a safe distance from environmental obstacles, this intervention strategy guides the robot toward the demonstration trajectory, thereby preventing collisions with surrounding obstacles.

Safety Recovery Mechanism To ensure that the system can promptly recover to a controllable and safe state when safety risks occur during training, we define two recovery points on the demonstration trajectory: *recover point*$_0$ and *recover point*$_1$. *Recover point*$_0$ is defined as a safe recovery target. When a safety risk occurs, motion planning is used to guide the robot from the current state to *recover point*$_0$. *Recover point*$_1$ is defined as a trajectory point where the robot is in stable contact with the interaction object. These two recovery points can be identified by replaying the demonstration trajectory on the robot. During training, at each time step, the Euclidean distance between the current end-effector pose and all end-effector poses along the entire demonstration trajectory is computed. The trajectory point with the minimum distance is selected and referred to as the minimum point of the full trajectory *cpoint*. The system

also maintains the minimum points over the most recent 20 time steps. If the difference between the current minimum point $cpoint_i$ and that from 20 time steps ago $cpoint_{i-20}$ is smaller than the threshold th_1, and $cpoint_i$ lies between $recover\ point_0$ and $recover\ point_1$ on the demonstration trajectory, the robot is considered unable to properly interact with the object. In this case, the system first uses motion planning to move the robot to $recover\ point_0$, and then replays the demonstration trajectory segment between $recover\ point_0$ and $recover\ point_1$ to complete the recovery process.

Intervention Termination Since the demonstration trajectory is not necessarily optimal, continuously applying intervention throughout training may reduce the policy's exploration capability and limit the advantages of reinforcement learning. Therefore, in experiments without initial state randomization, if the total number of interventions in an episode is less than 10 and the episode successfully completes the task, all subsequent interventions are disabled.

In all tasks considered in this paper, the threshold th_1 is set to 0.005 m, and the threshold th_2 is set to 0.02 m.

4 Experiments

We conduct real-world experiments to evaluate the effectiveness of AutoSERL, aiming to address the following three questions: (1) Does AutoSERL achieve higher efficiency compared to existing baselines? (2) What is the contribution of the sliding window intervention and the safety recovery mechanism to overall performance? (3) How well does AutoSERL perform under positional variations?

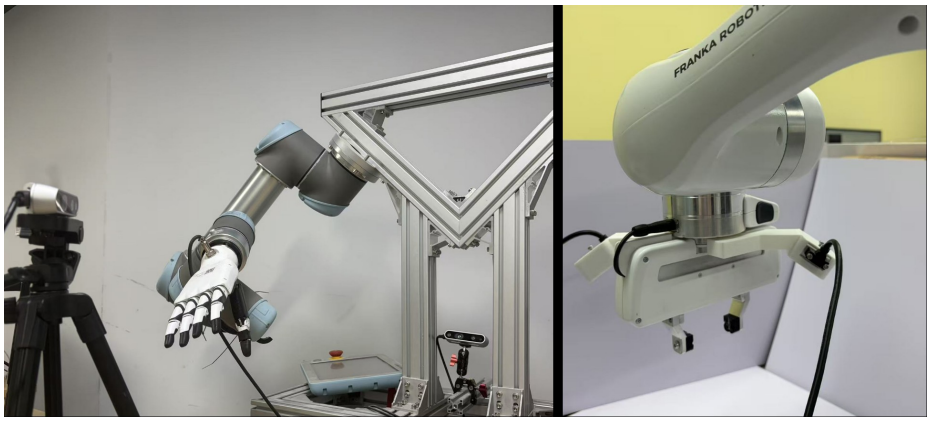

Fig. 2. Experimental setup. Left: the setup for the hanging and hinge-based tasks, consisting of a UR5 robot and two Intel RealSense D435 cameras. Right: the setup for the insertion tasks, consisting of a Franka robot and two wrist-mounted Intel RealSense D405 cameras.

4.1 Experimental Setup

Hardware and Protocol. The insertion tasks are conducted on a Franka robot arm with a parallel gripper and two wrist-mounted Intel RealSense D405 cameras. The hanging and hinge-based tasks are implemented on a UR5 robot equipped with an Inspire dexterous hand and two Intel RealSense D435 cameras. The overall setup is shown in Fig. 2. Across all tasks, the observation consists of RGB images from the two cameras and robot proprioceptive states. For the Franka platform, proprioception includes end-effector poses, velocity, forces/torques, and gripper state. For the UR5 platform, proprioception includes end-effector poses and forces/torques. The action space is a 6D delta end-effector pose for all tasks. Training uses manually annotated binary sparse rewards. Episodes terminate upon task success or after 300 time steps. During evaluation, all automatic intervention mechanisms are disabled, and each task is evaluated over 50 episodes.

Task Overview. We implement three categories of contact-intensive manipulation tasks, as illustrated in Fig. 3: insertion tasks (plug insertion and USB insertion), hanging tasks (hanging a correction tape, a hanger, and a spoon), and a hinge-based task (drawer pulling using a hook). These tasks are characterized by rich physical contact and low tolerance for execution errors, requiring high manipulation precision, accurate pose alignment, stable contact control, and strong robustness to disturbances. If the above conditions are not satisfied, the robot may easily become stuck, as shown in Fig.4.

The insertion tasks demand high-precision control in constrained contact settings. In both USB insertion and two-pin plug insertion, the initial distance between the gripper and the target configuration is approximately 15 cm. Success is defined as complete insertion of the object into the corresponding port or socket.

The hanging tasks require fine-grained control of both pose and trajectory to ensure correct geometric constraint interactions between the object's hole and the hook. The initial end-effector distance from the target hanging configuration is approximately 15 cm for the correction tape and spoon tasks, and approximately 20 cm for the hanger task. The task is considered successful when the hole of the correction tape or spoon, or the curved part of the hanger, establishes contact with the curved region of the hook.

The hinge-based task consists of drawer pulling. The robot must first hook the drawer handle and then pull the drawer outward by 5 cm. The initial distance between the hook and the handle is approximately 10 cm. Success is achieved when the drawer is displaced by 5 cm. The main challenge lies in maintaining stable contact between the hook and the handle after initial engagement, while the drawer moves under the kinematic constraint of the hinge joint.

4.2 Results and Analysis

Training Efficiency. We compare AutoSERL with SERL [8] under the same training time budget to evaluate training efficiency. For each task, SERL is

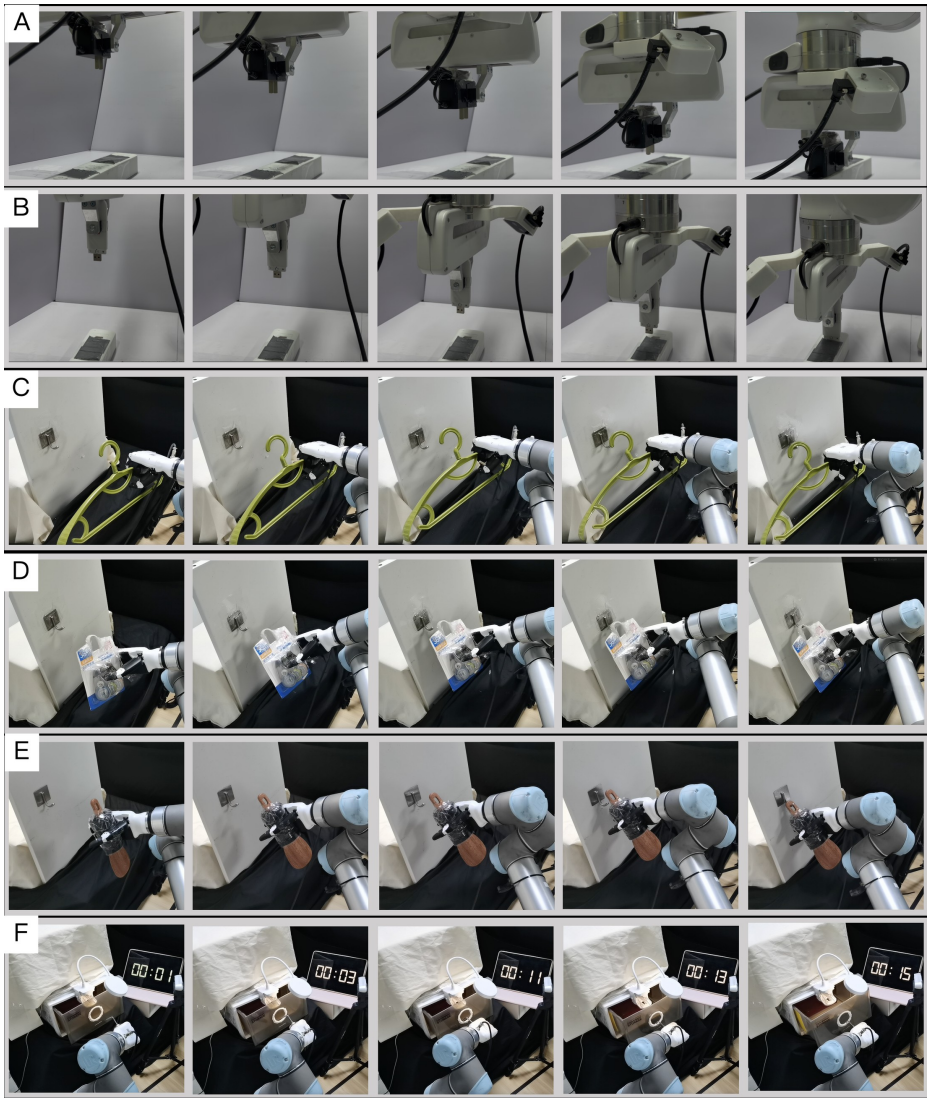

Fig. 3. Overview of the experimental tasks: (A)Plug Insertion. (B)USB Insertion. (C)Hanger Suspension. (D)Correction Tape Suspension. (E)Spoon Suspension. (F)Drawer Opening.

initialized with 20 demonstration trajectories. As shown in Table 1, AutoSERL achieves higher success rates than SERL under identical training duration, demonstrating improved sample efficiency. Fig. 5 presents the training curves for each task. The intervention steps curve shows that the number of automatic interventions gradually decreases as training progresses, indicating that the learned pol-

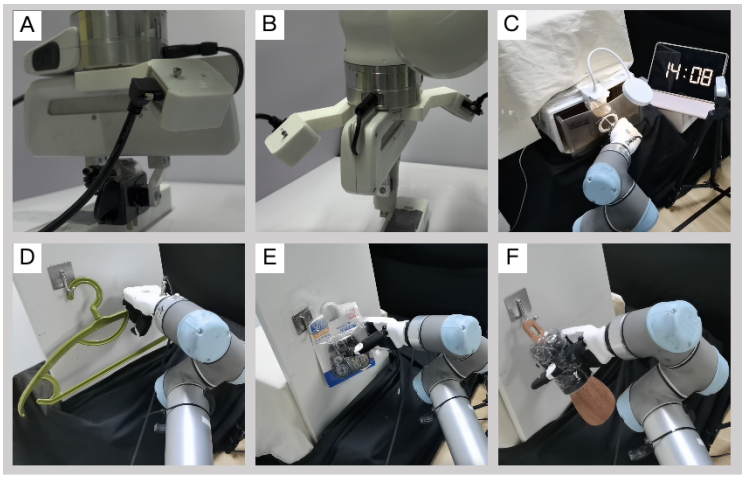

Fig. 4. Overview of the stuck cases across different tasks: (A)Plug Insertion. (B)USB Insertion. (C)Drawer Opening. (D)Hanger Suspension. (E)Correction Tape Suspension. (F)Spoon Suspension.

Table 1. Success rate of SERL and AutoSERL under the same training time budget.

Task	Training Time (min)	Success Rate	
		SERL	AutoSERL
USB Insertion	8	20/50	50/50
Plug Insertion	8	0/50	50/50
Hanger Suspension	33	0/50	50/50
Correction Tape Suspension	25	6/50	50/50
Spoon Suspension	35	0/50	50/50
Drawer Opening	45	0/50	50/50

Table 2. Success rate comparison between AutoSERL and baselines.

Task	AutoSERL	BC	MILES
USB Insertion	50/50	5/50	0/50
Plug Insertion	50/50	2/50	33/50
Correction Tape Suspension	50/50	38/50	1/50
Hanger Suspension	50/50	37/50	42/50
Spoon Suspension	50/50	0/50	2/50
Drawer Opening	50/50	35/50	0/50

icy increasingly approximates the demonstration behavior. The episode return curve shows that AutoSERL stably accomplishes the tasks without intervention after the same training duration, whereas SERL fails to achieve consistent task success.

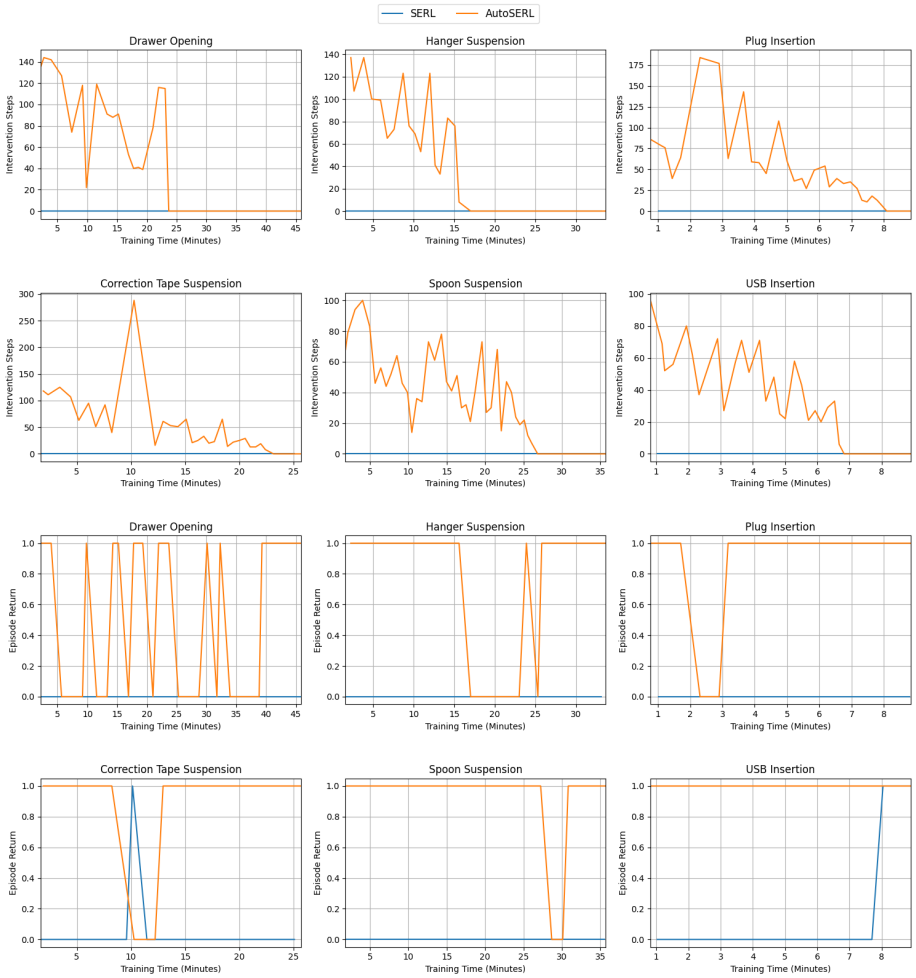

Fig. 5. Training curves of time versus intervention steps and time versus episode return for each task under SERL and AutoSERL.

Comparison with Baselines. To further validate the effectiveness of AutoSERL, we compare it with BC and MILES [13] in terms of final success rate. For BC, 1 demonstration is used for the hinge-based task, 10 demonstrations for the hanging tasks, and 20 demonstrations for the insertion tasks. For MILES, we adopt the same data augmentation randomization range as reported in the original work, namely ± 4 cm in translation and $\pm 4^\circ$ in rotation. The results are summarized in Table 2. AutoSERL consistently outperforms both baselines

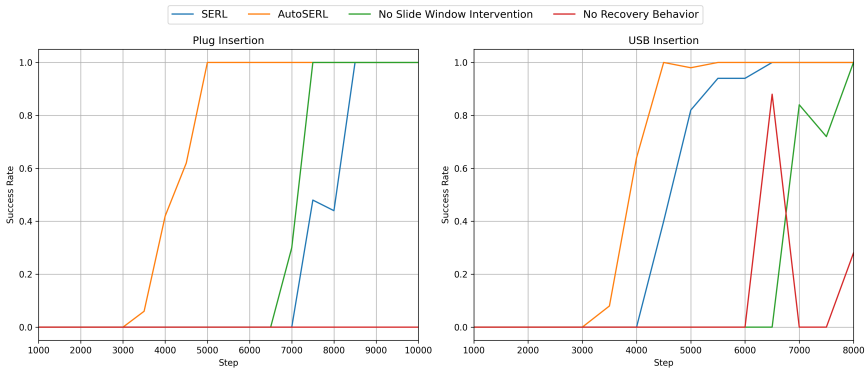

Fig. 6. Ablation study on the plug insertion and USB insertion tasks.

across all tasks, demonstrating its superior performance in real-world manipulation scenarios.

4.3 Ablation Studies

To evaluate the contribution of each component in AutoSERL, we conduct two ablation studies: (1) *No sliding window intervention*: the sliding window intervention mechanism is removed, and no intervention points are used to guide the robot during training. (2) *No recovery mechanism*: the safety recovery mechanism is removed, and when the robot encounters failure states, it must rely solely on its own exploration to recover.

We perform the ablation experiments on the plug insertion and USB insertion tasks. Fig. 6 presents the results. The full AutoSERL method reaches a 100% success rate the earliest. Removing the sliding window intervention increases the number of training steps required to reach 100% success. Removing only the recovery mechanism while retaining the sliding window intervention leads to worse performance than SERL, because the sliding window strategy may guide the robot into near-failure states. Without an explicit recovery mechanism, the policy must rely solely on sparse-reward exploration to escape these states, resulting in higher failure rates and less stable learning.

4.4 Evaluation under Positional Variations

We randomize the initial object position within a ± 3 cm range in the x–y plane to evaluate robustness to positional perturbations. Since positional variations alter the distribution of intervention triggers during training, all intervention mechanisms are kept active throughout the training phase of this experiment; evaluation follows the standard protocol with all interventions disabled. Under this setting, we compare SERL and AutoSERL using the same training configuration and report training-time versus success-rate curves for the plug insertion

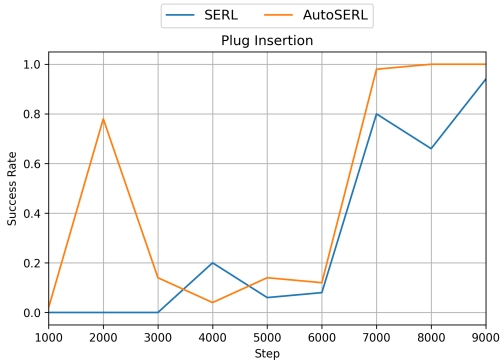

Fig. 7. Plug insertion under positional variations.

task. As illustrated in Fig. 7, AutoSERL achieves higher success rates and more stable convergence than SERL, demonstrating improved robustness to positional variations.

5 Limitations

Although AutoSERL shows strong performance in real-world manipulation tasks, several limitations remain. First, the proposed recovery mechanism is relatively simple and relies on motion planning, which may not always succeed when dealing with complex object geometries or cluttered environments. Developing more general recovery strategies, such as learning-based recovery policies, is an important direction for future work. Second, the method still requires a manually collected demonstration trajectory to initialize the guidance mechanism. Further reducing human involvement through automatic trajectory generation would improve the practicality of the approach.

6 Conclusion

We propose AutoSERL, a real-world reinforcement learning method that enables training through automatic intervention using only a single demonstration trajectory. AutoSERL incorporates sliding window intervention, safety recovery intervention, and intervention termination to ensure safe and stable real-world reinforcement learning. AutoSERL outperforms multi-demonstration RL, behavior cloning, and one-shot imitation learning baselines across six contact-intensive tasks spanning three task categories. In the future, AutoSERL can be combined with techniques such as automatic reward generation and automatic reset, enabling fully autonomous real-world reinforcement learning without human involvement.

References

1. Chen, Y., Tian, S., Liu, S., Zhou, Y., Li, H., Zhao, D.: Conrft: A reinforced fine-tuning method for vla models via consistency policy (2025), https://arxiv.org/abs/2502.05450
2. Dulac-Arnold, G., Mankowitz, D., Hester, T.: Challenges of real-world reinforcement learning. arXiv preprint arXiv:1904.12901 (2019)
3. Hoque, R., Balakrishna, A., Novoseller, E., Wilcox, A., Brown, D.S., Goldberg, K.: Thriftydagger: Budget-aware novelty and risk gating for interactive imitation learning (2021), https://arxiv.org/abs/2109.08273
4. Hu, K., Shi, H., He, Y., Wang, W., Liu, C.K., Song, S.: Robot trains robot: Automatic real-world policy adaptation and learning for humanoids (2025), https://arxiv.org/abs/2508.12252
5. Johns, E.: Coarse-to-fine imitation learning: Robot manipulation from a single demonstration (2021), https://arxiv.org/abs/2105.06411
6. Kelly, M., Sidrane, C., Driggs-Campbell, K., Kochenderfer, M.J.: Hg-dagger: Interactive imitation learning with human experts (2019), https://arxiv.org/abs/1810.02890
7. Liu, H., Nasiriany, S., Zhang, L., Bao, Z., Zhu, Y.: Robot learning on the job: Human-in-the-loop autonomy and learning during deployment (2023), https://arxiv.org/abs/2211.08416
8. Luo, J., Hu, Z., Xu, C., Tan, Y.L., Berg, J., Sharma, A., Schaal, S., Finn, C., Gupta, A., Levine, S.: Serl: A software suite for sample-efficient robotic reinforcement learning. In: 2024 IEEE International Conference on Robotics and Automation (ICRA). pp. 16961–16969. IEEE (2024)
9. Luo, J., Xu, C., Wu, J., Levine, S.: Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. Science Robotics **10**(105), eads5033 (2025)
10. Mandlekar, A., Xu, D., Martín-Martín, R., Zhu, Y., Fei-Fei, L., Savarese, S.: Human-in-the-loop imitation learning using remote teleoperation (2020), https://arxiv.org/abs/2012.06733
11. Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., Riedmiller, M.: Playing atari with deep reinforcement learning. arXiv preprint arXiv:1312.5602 (2013)
12. Nair, A., McGrew, B., Andrychowicz, M., Zaremba, W., Abbeel, P.: Overcoming exploration in reinforcement learning with demonstrations. In: 2018 IEEE international conference on robotics and automation (ICRA). pp. 6292–6299. IEEE (2018)
13. Papagiannis, G., Johns, E.: Miles: Making imitation learning easy with self-supervision. arXiv preprint arXiv:2410.19693 (2024)
14. Rajeswaran, A., Kumar, V., Gupta, A., Vezzani, G., Schulman, J., Todorov, E., Levine, S.: Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. arXiv preprint arXiv:1709.10087 (2017)
15. Ross, S., Gordon, G.J., Bagnell, J.A.: A reduction of imitation learning and structured prediction to no-regret online learning (2011), https://arxiv.org/abs/1011.0686
16. Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., et al.: Mastering the game of go with deep neural networks and tree search. nature **529**(7587), 484–489 (2016)

17. Vecerik, M., Hester, T., Scholz, J., Wang, F., Pietquin, O., Piot, B., Heess, N., Rothörl, T., Lampe, T., Riedmiller, M.: Leveraging demonstrations for deep reinforcement learning on robotics problems with sparse rewards. arXiv preprint arXiv:1707.08817 (2017)
18. Wu, P., Shentu, Y., Liao, Q., Jin, D., Guo, M., Sreenath, K., Lin, X., Abbeel, P.: Robocopilot: Human-in-the-loop interactive imitation learning for robot manipulation (2025), https://arxiv.org/abs/2503.07771
19. Xu, X., Hou, Y., Xin, C., Liu, Z., Song, S.: Compliant residual dagger: Improving real-world contact-rich manipulation with human corrections (2025), https://arxiv.org/abs/2506.16685